Cue-driven memory — requirement

Oct 14, 2026 · @Jesus Gonzalez

Replace Claude Code's local MEMORY.md with memory held in AgentOps and recalled automatically by what the session is doing, the way human recall is triggered by context rather than by choosing to look something up.

Problem

Sessions lose knowledge they already have, so you re-explain things and steer by hand. It fails in three ways, and all three show up in your sessions:

1. **Never recalled.** MEMORY.md is loaded, but nothing ties an entry to the moment it matters, so the agent never connects the two.
2. **Recalled but not followed.** The entry was in context and the agent acted against it anyway. These are mostly rules stored as if they were facts.
3. **Bypassed by plan mode.** Planning sends the session straight into the codebase with Read, Grep and Explore subagents. It never calls AgentOps, because using the MCP tools is a choice the agent makes and it doesn't make it.

The common cause is that recall depends on the agent deciding to remember. Human recall doesn't work that way: what you are looking at triggers it, whether you ask or not.

Evidence to attach before the spec: two or three session transcripts, one per failure mode, with `agentops usage report` output showing what was in context when the agent went wrong.

Outcome

When this ships, these are true:

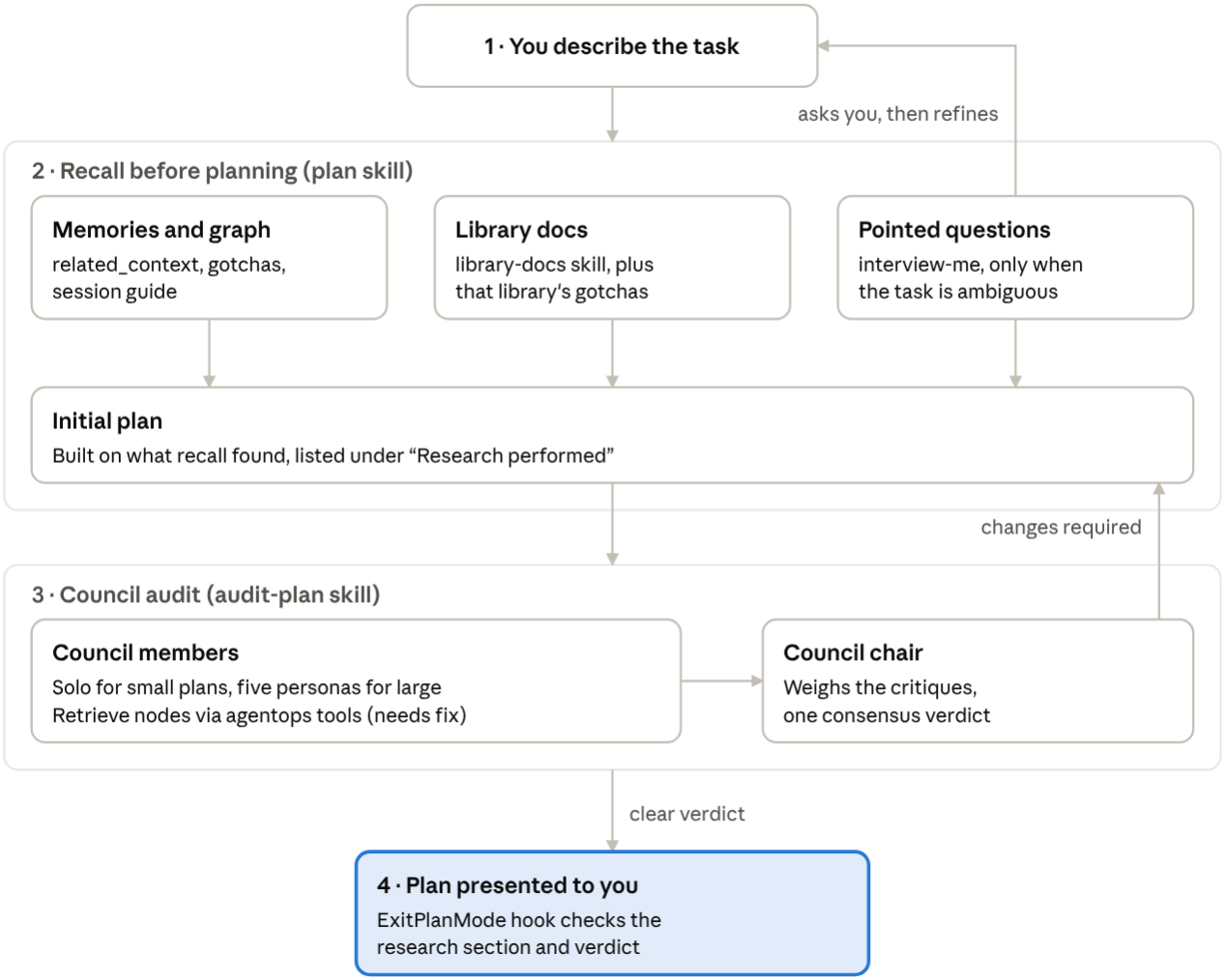
- A relevant memory reaches the context **without the agent calling any tool**, in normal mode and in plan mode, and in Explore subagents too if hooks reach them.
- A memory surfaces when its cue appears (a file read, a prompt mentioning a module, an error) and not otherwise. Injected memory stays inside a fixed budget per turn.
- Rules that must always hold are enforced or always present, never left to recall.
- Existing MEMORY.md files can be imported once, with every entry routed to the right memory kind and nothing silently dropped.
- Session learnings become memories without you asking: the agent writes during the session, and consolidation runs at session end.

- A memory that was recalled and turned out wrong gets flagged for review; one recalled and used gets stronger.
- **Measured:** on a replay of the evidence sessions, the memory that was missed now appears before the step where the agent went wrong.

Task-start flow

Every task starts the same way: recall and research first, a council audit second, and only then a plan for you. Nothing is edited before you approve.

No plan reaches you until recall and the council have run



task-start flow · 4 steps, 2 loops

The council members review the code directly, so they need the same retrieval as the planner. Each member looks up the nodes for the files it reads, instead of relying on the planner's summary.

What exists today, and the gap

Most of this flow is already written as skills in `agentops-core/prompts/`. Two kinds of gap remain: skills that run only when the agent chooses to use them, and one wiring bug.

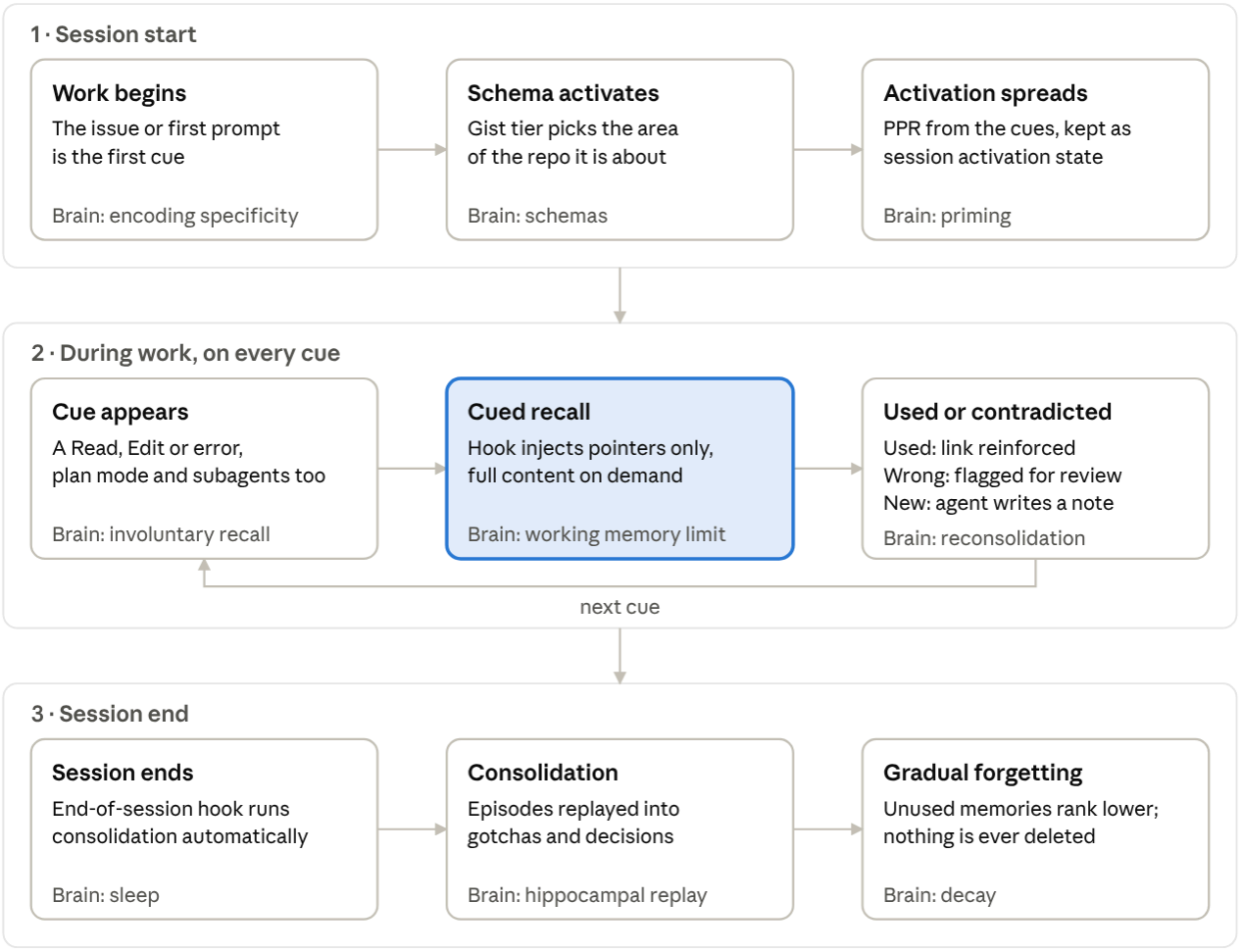
Step	Exists today	Gap
2. Recall before planning	The `plan` skill already requires `related_context`, `list_gotchas` and `get_session_guide` before finalizing, plus a "Research performed" section	The skill runs only if invoked. Claude Code's built-in plan mode never invokes it, which is the bypass you see.
2. Library docs	`library-docs` skill: `get_docs` / `search_docs`, then self-serve ingestion, then training knowledge flagged as a last resort	It never loads gotchas attached to the library. Docs and library gotchas should come back together.
2. Pointed questions	`interview-me` skill exists	`plan` does not call it. There is no rule for when the ambiguity is high enough to ask.
3. Council	`audit-plan` spawns `plan-council-member` (solo, or five personas) and `plan-council-chair`	**Bug:** members declare `tools: Read, Grep, Bash`. In Claude Code an explicit tools list excludes MCP tools that are not named, so the Completeness-checker cannot actually call `list_gotchas` / `related_context` as its persona says. `vault-archivist` shows the fix: name the `mcp__agentops__*` tools.
4. Present	Plan mode's `ExitPlanMode`	Nothing checks that steps 2 and 3 happened before the plan is shown.

Enforcement idea for the spec: a `PreToolUse` hook on `ExitPlanMode` that denies the call, with a reason, unless the plan carries a "Research performed" section and an audit verdict. A hook can't make the agent remember, but it can stop it from presenting a plan that skipped recall. This needs verifying against the hooks documentation.

How recall should behave

Recall happens in three phases, modeled on how a person picks up an issue. The highlighted step is the core of this feature; most of the others reuse parts AgentOps already has.

Recall is triggered by what the session does, not by the agent asking



memory lifecycle · 3 phases, 9 steps, 1 loop

Hooks deliver the middle phase, so recall never depends on the agent choosing to call a tool, and plan mode cannot route around it.

Memory kinds

A MEMORY.md mixes three kinds of memory that the brain handles in different systems. Import routes each entry by kind, because a rule stored as a fact is exactly the "recalled but not followed" failure.

Kind	Brain analogue	Example entry	Goes to	Delivered by
Semantic	Neocortex: facts and the reasons behind them	"The scanner skips vendored dirs because…"	Gotcha, decision or knowledge node, matched to symbols	Cue-triggered recall
Episodic	Hippocampus: what happened, when, in what context	"Last session we tried X and it broke Y"	Session events	Session guide; consolidated into semantic memory over time
Procedural	Basal ganglia: habits, not recall	"Always run `cargo deny` before committing"	AGENTS.md, a skill, or a hook	Always present, or enforced by a hook

An entry the importer cannot classify with confidence goes to a review queue in the Gotchas workspace. It is never guessed, and never dropped.

Scope, non-goals, constraints

In scope: Claude Code only, local and remote-connected repos, a one-time MEMORY.md import, and both write paths (agent writes mid-session plus consolidation at session end).

Non-goals:

- Cursor, Codex, Gemini or any other tool. Revisit only if their hook support catches up.
- Automatic deletion. Decay lowers recall strength; it never removes a memory, consistent with ADR 0016 (curate, don't resolve).
- Replacing CLAUDE.md or AGENTS.md. Procedural memory still lives there.
- New ranking math. Reuse `effective_edge_weight`, personalized PageRank and the gist-then-detail tiers.

Constraints from existing decisions:

- Injection lives inside the session guide's character budget, or in a separate budget the spec sets. Pointers first, full content on demand.
- Agent memory writes must work in the default `advisor` mode, so they need the same kind of exception `add_note` has (ADR 0007).
- Remote-connected repos write through MCP only, never local files (`docs/vault-protocol.md`).

- No `PreCompact` hook, unless consolidation's needs change ADR 0035's reasoning.

Verification findings

Two of the three hook assumptions hold; the plan-mode gate does not. Checked against the Claude Code hooks reference, public issue reports, and `agentops-cli/src/hook_capture.rs`.

Question	Answer	Consequence
Can a hook tell it is in plan mode?	**Yes, probably.** Every hook gets a common `permission_mode` field, with `"plan"` as a value. The docs add that not every event receives it.	`UserPromptSubmit` branches on `permission_mode == "plan"`. Confirm the field arrives on that event in your Claude Code version.
Do hooks fire inside subagents?	**Yes.** `PreToolUse` / `PostToolUse` fire in subagents and carry `agent_id` and `agent_type`. `SubagentStart` can add context before the subagent's first turn but cannot block. Subagent frontmatter can also declare hooks.	Explore subagents in plan mode and the council members get cued recall without calling tools. `SubagentStart` can hand each council member the plan's research.
Can a hook block `ExitPlanMode`?	**No, not reliably.** The docs never mention it. Issue reports say `PreToolUse` doesn't fire for Enter/ExitPlanMode, a `deny` is ignored, and a `PermissionRequest` allow is ignored since v2.1.199.	Drop the hard gate from milestone 1. Enforce the flow softly instead.
Where should recalled memory go?	`PostToolUse` `additionalContext` lands next to the tool result. `UserPromptSubmit` context goes alongside the prompt (capped at 10,000 characters, per the repo's own comment).	Cued recall on file reads uses `PostToolUse`. The task-start flow uses `UserPromptSubmit`.

Question	Answer	Consequence
How costly is a hook call?	Each event spawns a fresh `agentops hook-capture` process that opens the store. `UserPromptSubmit` times out at 30 s by default, and on timeout its context is discarded.	Per-read recall can't load the ONNX embedding model each time. Hooks need a cheap path.
Is any of this already built?	Six hooks are installed by `connect --with-hooks`. `UserPromptSubmit` already re-injects pinned notes every turn. Hooks read only the local store.	Milestone 1 extends the existing handlers. Remote-connected repos currently get no hook recall.
Can consolidation run at session end?	`SessionEnd` hooks share a 1.5 s budget, raisable to 60 s with `timeout`. The existing handler already spawns a detached `usage sync`.	Consolidation uses the same detached pattern. ADR 0035 is unaffected.

Found while checking: hooks ignore the remote server. `connect --remote --with-hooks` installs the same hooks as local mode. But `hook-capture` opens the store through `agentops_store_open::open_store`, which uses `AGENTOPS_DATABASE_URL` or else `.context/graph.db` on your machine. It never reads `.context/agentops-remote.json`. On a server-connected repo, session events and pinned notes go to an orphaned local graph, and the session guide is built from it. MCP over `/mcp` works; the hooks don't. The server has no REST route for session events, so the fix needs one, or hooks calling `/mcp`. Only `usage sync` handles remote today.

Sources: Claude Code hooks reference · Issue: plan mode transitions invisible to hooks (#21282) · Issue: PreToolUse deny ignored on ExitPlanMode · Issue: PermissionRequest allow ignored since v2.1.199 · Tip: PermissionRequest hook matched on ExitPlanMode

Open questions

- [ ] Do `PreToolUse` / `PostToolUse` hooks fire for tool calls inside Explore and other subagents, and can their output reach the subagent's context? This decides whether the plan-mode fix holds. Verify against current Claude Code docs.
- [ ] Is hook latency acceptable? A recall lookup on every Read must stay fast enough not to slow the session; set a target in the spec.
- [ ] What counts as a cue: file path, symbols in the file, words in the prompt, error text? Which ones go in a first version?

- [ ] How is "recalled and used" versus "recalled and ignored" detected without an LLM call per turn?
- [ ] Activation state: does it live in `session_events`, or in a new per-session table?
- [ ] Should recall suppress repeats, so the same gotcha is not injected on every read of the same file in one session?
- [ ] Import: which MEMORY.md locations are read (per-project, user-level), and does the original file stay as a fallback during the trial?

Decided for the task-start flow:

- **Plan mode always runs the full flow.** Entering plan mode means there is work to do, so the first prompt in plan mode starts at step 1 every time, with no size check. To verify: that the `UserPromptSubmit` hook input reports the session's permission mode, so the hook can tell it is in plan mode.
- **Council members get the full agentops read toolset.** They are private subagents, so blocking the flow costs more than a wide toolset. Each `mcp__agentops__*` tool is named in the members' `tools:` list.
- **Pointed questions have a trigger rule and a cap.** The spec defines both: when the task is ambiguous enough to ask, and the maximum number of questions before the agent plans with its stated assumptions.

Decided after verification:

- **Soft enforcement of the flow.** The plan-mode prompt hook injects the flow. The plan must show "Research performed" and the council verdict, and a `Stop` hook flags a plan that lacks them. No hard gate on `ExitPlanMode`.
- **The server comes first.** You run both a local and a server instance, and you're focusing on the server. Hooks must follow `.context/agentops-remote.json` like the vault protocol does, so fixing hook routing is part of milestone 1, not a follow-up.
- **Hooks recall through the graph and lexical search only.** Exact-name, BM25 and file/symbol edges. No embedding model in hook calls; semantic search stays in the planning step.